# **Title**: Region-Aware Event Assignment System

---

## 📜 Problem Statement

You are building a backend service for a **moderation platform**. The platform ingests **events** tagged with regions (e.g., Asia, Europe, US) and allows **moderators** to view and claim those events for review.

Each event:

- Belongs to a **region**
- Can only be claimed by a **moderator from the same region**
- Once claimed, is **locked for 15 minutes**
- If not acknowledged within 15 minutes, the event becomes **claimable again**

---

## 🎯 Requirements

### 🧾 Functional Requirements

1. **Moderator Login**
   - A moderator logs in by providing their `user_id` and `region`
2. **View Available Events**
   - A logged-in moderator can see unassigned, unexpired events for **their region only**
3. **Claim an Event**
   - A moderator can claim an available event
   - The event is locked for 15 minutes from the time of assignment
4. **Acknowledge an Event**
   - Within 15 minutes, a moderator can acknowledge that they've started reviewing it
5. **Automatic Expiry/Reassignment**
   - If an event is not acknowledged within 15 minutes, it becomes available again for others to claim

---

## Event Source

*Note: The content of this document is confidential and intended for the recipient specified in message only. It is strictly forbidden to share any part of this message with any third party, without a written consent of the sender. The content scenario is a hypothetical situation for the purposes of this exercise only.*

You are free to choose how to **simulate or mock** the event ingestion.

Possible options (your choice):

- Load/Poll events from a mock Regional Database(s)
- Load/Poll events from a **file system** (e.g., `events/asia/event1.json`)

  Or any other options that you deem suitable

Please document your approach, rationale and future enhancements clearly in the README.

---

## APIs (suggested)

Feel free to design the API in your own way, but the following endpoints are suggested:

| Endpoint | Purpose |
|---|---|
| Login User | Simulate moderator login |
| `Get Available Events` | List claimable events for logged-in user |
| `Claim Events` | Claim an event(s) for region |
| `Acknowledge event` | Acknowledge processing of claimed event |

---

*Note: The content of this document is confidential and intended for the recipient specified in message only. It is strictly forbidden to share any part of this message with any third party, without a written consent of the sender. The content scenario is a hypothetical situation for the purposes of this exercise only.*

# Expected Behavior

- Moderators should **only** see and claim events for **their own region**
- An event can be claimed by **only one moderator at a time**
- Claimed events should **expire after 15 minutes** if not acknowledged
- The system should support **reassignment** of expired events
- The Logged In Users should be continuously supplied with events as they keep acknowledging/processing events

# Evaluation Focus

We'll evaluate submissions on the following dimensions:

### API Design

We're looking for clean, RESTful APIs with properly scoped routes and clear semantics.

### Data Modeling

Your solution design should have a thoughtful separation between events, moderators, and assignments. Normalize where necessary and ensure extensibility.

### Time-Bound Logic

You must handle the 15-minute expiry and reassignment of unacknowledged events. The implementation should be testable and correct under concurrent conditions.

### Code Quality

We value clean, modular, and readable code. Bonus points for testability, separation of concerns, and documentation where helpful.

### Working MVP

Your submission must be a working end-to-end solution. It should support:

- Moderator login
- Viewing region-specific, unassigned events
- Claiming an event
- Acknowledging within 15 minutes
- Automatic expiry and reassignment of unacknowledged events

*Note: The content of this document is confidential and intended for the recipient specified in message only. It is strictly forbidden to share any part of this message with any third party, without a written consent of the sender. The content scenario is a hypothetical situation for the purposes of this exercise only.*

### Dockerization

Your entire solution should run via Docker. A `docker-compose up` should spin up the backend, frontend (if included), and any supporting services like databases or queues.

### Minimal Frontend

**Include a minimal UI to demonstrate the full flow:** logging in **(Can be mocked with hardcoded user)**, viewing events, claiming one, and acknowledging. It can be very simple — focus on usability over styling.

---

### Extra Credit

We'll also consider these bonus areas:

- Exposing useful metrics like total assigned, currently active, and expired events
- Using in-memory caching for performance (e.g., frequently accessed queries, limits per user)
- Implementing background workers or scheduled jobs for expiring stale assignments
- Implementing SSE/Sockets for event distribution.
- Writing basic unit or integration tests for critical logic

---

## 📦 Final Deliverables

Please share the following:

### 1. Source Code

- A full working project (backend + minimalistic frontend)
- Should be dockerized and runnable with a single command: `docker-compose up`

### 2. README

Include:

- Setup instructions
- Description of your approach to event ingestion (DB, file system, or otherwise)
- API documentation (Postman collection, curl commands, or clear endpoint list)
- Any design decisions, assumptions, or tradeoffs
- Loom Video explaining the solution

*Note: The content of this document is confidential and intended for the recipient specified in message only. It is strictly forbidden to share any part of this message with any third party, without a written consent of the sender. The content scenario is a hypothetical situation for the purposes of this exercise only.*

**Note:**

- There are no restrictions on using any boilerplates, libraries or even using LLM's or LLM powered IDE's like Cursor.
- Minimum Requirements for a submission is to have a prototype instance running for event distribution.
- Please feel free to take any necessary assumptions for implementing the solution.
- We understand that the problem statement may have a scope for a detailed implementation and request to use your time to efficiently create a solution that **delivers maximum impact within the given time frame** and please feel free to document future proposals for Peer to Peer review round.

*Note: The content of this document is confidential and intended for the recipient specified in message only. It is strictly forbidden to share any part of this message with any third party, without a written consent of the sender. The content scenario is a hypothetical situation for the purposes of this exercise only.*